3D MODEL

View online: https://www.construct.net/en/make-games/manuals/construct-3/scripting/scripting-reference/plugin-interfaces/3d-model

The `I3DModelInstance` interface derives from IWorldInstance to add APIs specific to the 3D model plugin.

3D Model APIs

loadModel(model, mesh, animation, playing, progress)

Load a new 3D model object. The `"model"` argument is required and is a string with the name of an existing 3D model object. `"mesh"` is an optional string and defaults to an empty string, if provided that mesh will be the only one being drawn, otherwise all meshes in the model will be drawn. `"animation"` is an optional string and default to an empty string, if not provided the first animation in the model will be used. `"playing"` is an optional boolean and defaults to *"false"*, if provided the model will start playing an animation as soon as possible. `"progress"` is an optional number and defaults to *"0"*. It accepts a value in the range **[0 - 1]** that indicates where the current animation will start playing from. The method returns a promise that resolves then the loading is complete.

> *The `"mesh"` parameters also accepts a **String Array** with all the mesh names that should be drawn as well as a **Map** where the keys are the names of the meshes and the values, **Booleans** indicating if the corresponding mesh should be drawn or not. Any mesh name not included in the **Array** or **Map** will default to being hidden.*

onLoad

This function is not defined by default. When defined it is executed after loading of a new 3D model object is complete.

onError

This function is not defined by default. When defined it is executed if there is any problem during the process of loading a new 3D model object.

loadTextureFromURL

Load a texture from a url into a mesh. The *"target"* parameter indicates which instances will be affected, *"object-type"* will affect all instances of the same object type while *"instance"* will only affect the instance used to call this function. The function returns a promise that resolves when the texture is loaded and is rejected if loading fails.

unloadTextures

Unload dynamically loaded textures from the provided 3D model and mesh. The function returns a promise that resolves when unloading completes and is rejected if there is any problem unloading.

modelName

returns the value of the current 3D model object in use. When set it acts as a shortcut for **loadModel** where only a 3D model name is provided.

meshNames

returns an array with all the meshes that are currently enabled. When set all the passed in mesh names will be enabled and any other meshes will be disabled.

animationName

returns the current animation. When set, the animation changes to the start of the new animation and doesn't play.

animationProgress

returns the progress of the current animation. When set it updates the progress of the current animation.

isPlaying

returns the playing state of the current animation. Setting will resume playback or stop it.

isLooping

sets or gets the current looping state of all animations in the model.

meshRenderMode

returns or changes the current render mode. It accepts these values: `"hierarchy"` to draw the enabled meshes as they where authored in the model file and `"isolate"` to draw the enabled meshes as if they where the only ones in the model file.

backfaceCulling

returns or changes the current state of back face culling.

onAnimationFinished

if set this callback executes when an animation finishes, it has one string argument which is the name of the animation that finished playing.

---

onAnimationLooped

if set this callback executes when an animation loops, it has one string argument which is the name of the animation that loopped.

---

offsetX

offsetY

offsetZ

return or set the current offset of the 3D model in relation to the position of the corresponding instance.

---

scaleX

scaleY

scaleZ

return or set the current scale of the 3D model. These values are compounded with the scale of the corresponding instance.

---

setTransform(x, y, z, type)

addTransform(x, y, z, type)

subTransform(x, y, z, type)

mulTransform(x, y, z, type)

divTransform(x, y, z, type)

These methods are used to apply changes on the three axes of a 3D model. `"x"` `"y"` and `"z"` are the main values that will be applied and `"type"` refers to the properties that will be affected, can be either *"offset"* *"rotation"* or *"scale"*.

---

animationDuration(animation)

returns the duration in seconds of the provided animation.

---

getAllMeshes

returns an array of strings with all the meshes in the current 3D model.

---

getAllAnimations()

returns an array of strings with all the animations in the current 3D model.

setMeshEnabled(mesh, enable)

Enable or disable the provided mesh.

setAllMeshesEnabled(enable)

Enable or disable all the meshes.

isMeshEnabled(mesh)

returns `"true"` or `"false"` depending on if the passed in mesh is enabled or not.

areAllMeshesEnabled()

returns `"true"` or `"false"` depending on if all the meshes are enabled or not.

meshExists(mesh)

returns `"true"` or `"false"` if the passed in mesh exists in the current 3D model or not.

play(animationName, progress): void

Play an animation. `"animationName"` is an optional string and default to an empty string, if not provided or if the name does not correspond to any animation, the method will affect the current animation. `"progress"` is an optional number and defaults to *"0"*. It accepts a value in the range **[0 - 1]** that indicates where the animation will start playing from.

stop()

Stop the current animation, resets progress to 0.

pause()

Pause the current animation, progress is unchanged.

resume()

Resume a paused animation from the current progress.